1. Write a function to remove duplicates from an array.

Approach 1: Using ES6 Set

```
function removeDuplicates(arr) {
  return [...new Set(arr)];
}
```

Approach 2: Using `filter()` and `indexOf()`

```
function removeDuplicates(arr) {
  return arr.filter((item, index) => arr.indexOf(item) === index);
}
```

Example:

```
const numbers = [1, 2, 2, 3, 4, 4, 5];
console.log(removeDuplicates(numbers)); // Output: [1, 2, 3, 4, 5]
```

2. Merge two arrays into one.

Approach 1: Using spread operator

```
const mergedArray = [...array1, ...array2];
```

Approach 2: Using `concat()` method

```
const mergedArray = array1.concat(array2);
```

Example:

```
const array1 = [1, 2, 3];
const array2 = [4, 5, 6];

console.log([...array1, ...array2]); // Output: [1, 2, 3, 4, 5, 6]
console.log(array1.concat(array2)); // Output: [1, 2, 3, 4, 5, 6]
```

3. Write a function to flatten a nested array.

Approach 1: Using built-in `flat()` method

```
function flattenArray(arr) {
  return arr.flat(Infinity);
}
```

Approach 2: Using recursion with `reduce()` and `concat()`

```
function flattenArray(arr) {
  return arr.reduce(
    (acc, item) =>
      Array.isArray(item) ? acc.concat(flattenArray(item)) : acc.concat(item),
    []
  );
}
```

Example:

```
const nestedArray = [1, [2, [3, 4], 5], 6];
console.log(flattenArray(nestedArray)); // Output: [1, 2, 3, 4, 5, 6]
```

4. Find the maximum and minimum values in an array.

Approach 1: Using `Math.max()` and `Math.min()` with spread operator

```
const max = Math.max(...arr);
const min = Math.min(...arr);
```

Approach 2: Using `forEach()`

```
let max = arr[0];
let min = arr[0];

arr.forEach((num) => {
  if (num > max) max = num;
  if (num < min) min = num;
});
```

Example:

```
const arr = [15, 7, 20, 3, 45];
console.log("Max:", Math.max(...arr)); // Output: 45
console.log("Min:", Math.min(...arr)); // Output: 3
```

5. Write a function to return the second largest number in an array.

Approach 1: Single pass (no sorting)

```
function findSecondLargest(arr) {
  let largest = -Infinity;
  let secondLargest = -Infinity;

  for (let num of arr) {
    if (num > largest) {
      secondLargest = largest;
      largest = num;
    } else if (num > secondLargest && num < largest) {
      secondLargest = num;
    }
  }

  return secondLargest === -Infinity ? null : secondLargest;
}
```

Approach 2: Using sorting and removing duplicates

```
function findSecondLargest(arr) {
  const uniqueArr = [...new Set(arr)];
  if (uniqueArr.length < 2) return null;
  uniqueArr.sort((a, b) => b - a);
  return uniqueArr[1];
}
```

Example:

```
const numbers = [12, 35, 1, 10, 34, 1];
console.log(findSecondLargest(numbers)); // Output: 34
```

6. Write a function to count the occurrences of each element in an array.

Approach 1: Using `forEach()` and object

```
function countOccurrences(arr) {
  const counts = {};
  arr.forEach((item) => {
    counts[item] = (counts[item] || 0) + 1;
  });
  return counts;
}
```

Example:

```
const fruits = ["apple", "banana", "apple", "orange", "banana", "grape"];
console.log(countOccurrences(fruits));
// Output: { apple: 2, banana: 2, orange: 1, grape: 1 }
```

7. Implement a function to rotate an array by n positions (left rotation).

Approach 1: Using `splice()` and `push()`

```
function rotateArray(arr, n) {
  const length = arr.length;
  n = n % length; // handle cases when n > arr.length
  arr.push(...arr.splice(0, n));
  return arr;
}
```

Approach 2: Using array reversal method

```
function reverse(arr, start, end) {
  while (start < end) {
    [arr[start], arr[end]] = [arr[end], arr[start]];
    start++;
    end--;
  }
}

function rotateArray(arr, n) {
  const length = arr.length;
  n = n % length;

  reverse(arr, 0, n - 1);
  reverse(arr, n, length - 1);
  reverse(arr, 0, length - 1);
  return arr;
}
```

Example:

```
const array = [1, 2, 3, 4, 5];
console.log(rotateArray(array, 2)); // Output: [3, 4, 5, 1, 2]
```

8. Write a function to get the union of two arrays.

Approach 1: Using `Set` and spread operator

```
function union(arr1, arr2) {
  return [...new Set([...arr1, ...arr2])];
}
```

Approach 2: Using `concat()` and `filter()`

```
function union(arr1, arr2) {
  return arr1
    .concat(arr2)
    .filter((item, index, array) => array.indexOf(item) === index);
}
```

Example:

```
const a = [1, 2, 3];
const b = [3, 4, 5];
console.log(union(a, b)); // Output: [1, 2, 3, 4, 5]
```

9. Write a function to get the intersection of two arrays.

Approach 1: Using `Set` and `for...of` loop

```
function getIntersection(arr1, arr2) {
  const set1 = new Set(arr1);
  const intersection = [];

  for (let item of arr2) {
    if (set1.has(item)) {
      intersection.push(item);
    }
  }

  return intersection;
}
```

Approach 2: Using `filter()` and `includes()`

```
function getIntersection(arr1, arr2) {
  return arr1.filter((item) => arr2.includes(item));
}
```

Example:

```
const array1 = [1, 2, 3, 5, 9];
const array2 = [1, 3, 5, 8];
console.log(getIntersection(array1, array2)); // Output: [1, 3, 5]
```

10. Remove all falsy values from an array (`false` , `0` , `""` , `null` , `undefined` , `NaN`).

Approach 1: Using `filter()` with `Boolean` constructor

```
function removeFalsyValues(arr) {
  return arr.filter(Boolean);
}
```

Approach 2: Using `filter()` with explicit callback

```
function removeFalsyValues(arr) {
  return arr.filter((item) => item);
}
```

Example:

```
const array = [0, 1, false, 2, "", 3, null, undefined, NaN];
console.log(removeFalsyValues(array)); // Output: [1, 2, 3]
```

11. Implement array shuffle.

Approach 1: In-place Fisher-Yates shuffle

```
function shuffle(array) {
  for (let i = array.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [array[i], array[j]] = [array[j], array[i]];
  }
  return array;
}
```

Approach 2: Pure function (no mutation)

```
function getShuffledArray(array) {
  const newArray = array.slice();

  for (let i = newArray.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [newArray[i], newArray[j]] = [newArray[j], newArray[i]];
  }

  return newArray;
}
```

Example:

```
const arr = [1, 2, 3, 4, 5];
console.log(shuffle(arr)); // Output: Shuffled array (in-place)
console.log(getShuffledArray(arr)); // Output: Shuffled array (new copy)
```

12. Sort Array of 0s, 1s, and 2s In-Place

Approach 1: Dutch National Flag Algorithm (One pass, O(n) time, O(1) space)

- Maintain three pointers: `low`, `mid`, and `high`.
- Swap values to put 0s at front, 2s at end, 1s in the middle.

```
function sort012(arr) {
  let low = 0,
    mid = 0;
  let high = arr.length - 1;

  while (mid <= high) {
    if (arr[mid] === 0) {
      [arr[low], arr[mid]] = [arr[mid], arr[low]];
      low++;
      mid++;
    } else if (arr[mid] === 1) {
      mid++;
    } else {
      // arr[mid] === 2
      [arr[mid], arr[high]] = [arr[high], arr[mid]];
      high--;
    }
  }

  return arr;
}
```

Approach 2: Using Counting (Two passes, but in-place)

1. Count the number of 0s, 1s, and 2s.
2. Overwrite the original array with counted numbers.

```
function sort012Counting(arr) {
  let count0 = 0,
    count1 = 0,
    count2 = 0;

  for (let num of arr) {
    if (num === 0) count0++;
    else if (num === 1) count1++;
    else count2++;
  }

  let i = 0;
  while (count0--) arr[i++] = 0;
  while (count1--) arr[i++] = 1;
  while (count2--) arr[i++] = 2;

  return arr;
}
```

Example:

```
const arr = [2, 0, 1, 2, 1, 0, 0, 2];

console.log(sort012Counting([...arr])); // Output: [0, 0, 0, 1, 1, 2, 2, 2]
console.log(sort012([...arr])); // Output: [0, 0, 0, 1, 1, 2, 2, 2]
```

13. Chunk an array into smaller arrays of a given size.

Approach 1: Using `slice()` in a for loop

```
function chunkArray(array, size) {
  const chunkedArr = [];
  for (let i = 0; i < array.length; i += size) {
    chunkedArr.push(array.slice(i, i + size));
  }
  return chunkedArr;
}
```

Example:

```
const data = [1, 2, 3, 4, 5, 6, 7];
console.log(chunkArray(data, 3)); // [[1,2,3],[4,5,6],[7]]
console.log(chunk(data, 2)); // [[1,2],[3,4],[5,6],[7]]
```

14. Find the maximum subarray sum.

Approach 1: Kadane's Algorithm (Optimal)

```
function maxSubarraySum(arr) {
  let maxSoFar = arr[0];
  let maxEndingHere = arr[0];

  for (let i = 1; i < arr.length; i++) {
    maxEndingHere = Math.max(arr[i], maxEndingHere + arr[i]);
    maxSoFar = Math.max(maxSoFar, maxEndingHere);
  }

  return maxSoFar;
}
```

Approach 2: Brute force nested loop

```
function maxSubarraySum(arr) {
  let maxSum = -Infinity;

  for (let i = 0; i < arr.length; i++) {
    let currentSum = 0;
    for (let j = i; j < arr.length; j++) {
      currentSum += arr[j];
      maxSum = Math.max(maxSum, currentSum);
    }
  }

  return maxSum;
}
```

Example:

```
const arr = [-2, -3, 4, -1, -2, 1, 5, -3];
console.log(maxSubarraySum(arr)); // Output: 7 (for subarray [4, -1, -2, 1, 5])
```

15. Find All Pairs in an Array That Sum to a Given Value

Approach 1: Using a Set for efficient lookup

```
function findPairsWithSet(arr, targetSum) {
  const seen = new Set();
  const pairs = [];

  for (let num of arr) {
    const complement = targetSum - num;
    if (seen.has(complement)) {
      pairs.push([complement, num]);
    }
    seen.add(num);
  }

  return pairs;
}
```

Approach 2: Brute Force (Nested loops)

```
function findPairsBruteForce(arr, targetSum) {
  const pairs = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === targetSum) {
        pairs.push([arr[i], arr[j]]);
      }
    }
  }
  return pairs;
}
```

Example:

```
const array = [1, 5, 7, -1, 5];
const target = 6;

console.log(findPairsBruteForce(array, target));
// Output: [ [1, 5], [1, 5], [7, -1] ]

console.log(findPairsWithSet(array, target));
// Output: [ [1, 5], [7, -1], [1, 5] ]
```